

第二天

场景

前端传过来一个表单数据，用什么类型接收？

管理系统中的数据列表用什么存储？

微信消息列表用什么存储？中途删除/撤回消息会不会同步到后端去？如果要同步到后端去有没有办法优化？

登录信息用什么存储？

防重复怎么设计？

MQ消息队列用什么实现？

常见数据结构

数组

哈希表

单向链表

双向链表

二叉树

红黑树

集合

用法

场景

前端传过来一个表单数据，用什么类型接收？

Map，HashMap。无序的就行了。DTO

管理系统中的数据列表用什么存储？

ArrayList，有序、快、允许重复

微信消息列表用什么存储？中途删除/撤回消息会不会同步到后端去？如果要同步到后端去有没有办法优化？

- ☒ LinkedHashMap: 有顺序
- ☒ LinkedList: 不用分配初始值大小
- ☒ ArrayList: 顺序、快、允许重复
- 删除操作: 不用同步
- 撤销操作: 需要同步
- 优化: 撤回的消息做个标记“back”, “wx:spec:__deleted”删除
- 问题: 分页后可能新页数据为空。前端处理或者不做修复, 按照需求来。不修复可以理解为用BUG换性能。

登录信息用什么存储?

- Map, HashMap, 无序、够快、内存开销小
- Redis
- 一级缓存 (内存, 热门用户) + 二级缓存 (Redis) + 三级缓存 (数据库)
- 一级缓存怎么实现?
 - ☒ HashMap: 无序、快、开销小
 - ☒ LinkedHashMap: 调整访问顺序, 增加淘汰策略, 保证数据是活跃的才留在内存中
 - 服务重启, 内存释放了。所以如果说真的要去加一级缓存, 就得用开源的主流的一级缓存框架去实现。以及缓存框架会基于序列化和反序列化, 将内存当中的数据序列化到磁盘当中去, 这样在重启的时候, 可以将序列化的数据反序列化到内存当中。

防重复怎么设计?

防重复指的是相同的参数提交了多次吧。

- ☒ 不可重复、无序、速度快: HashSet、HashMap (key=参数, value=请求次数)
- ☒ ConcurrentHashMap: 支持并发, 上一种并发下无法解决
- 时间过期问题: 插入的时候, 或者访问的时候, 遍历一下, 根据时间来动态删除元素

MQ消息队列用什么实现?

Queue队列

常见数据结构

数组

动画演示：[数组内存.html](#)

数组在内存中是连续的，目的就是为了更快的访问。需要注意的是，数组中如果存对象，那么数组的连续内存保存的是对象的引用，也就是引用在内存中是连续的。

缺点

- 头部和中间插入慢：需要移动位置。例如A -> C -> D -> E -> F，中间插入B，那么从C到F都需要移动位置，等同于插队。
- 头部和中间删除慢：需要移动位置
- 大小固定
- 预分配太大容易浪费内存，太小又容易溢出

优点

- 访问速度快：因为内存是连续的，所以根据索引可以快速访问到内存数据
- 修改速度快：只需要找到索引然后调整引用或值

哈希表

把一个对象计算成一个数字，通过数字来决定这个对象放在数组当中的哪个坐标中。

单向链表

链表的内存地址不是连续的。因为链表中的对象一个一个都是独立对象，在创建对象时，压根不知道他们会是一个链表结构。只是在每个元素中增加了prev或者next来引用上一个对象或者下一个对象。

链表的索引/下标：是实时数出来的，数组是提前分配的。

缺点

- 单向遍历：只能从头到尾遍历
- 删除尾节点慢：需要先遍历，找到倒数第二个节点，然后将next标记为null
- 删除任意节点慢：需要先遍历，找到上一个或者下一个，然后改变其引用，例如A -> B -> C，删除B时，需要遍历找到A，然后将A的next调整为B的next（即C）
- 随机获取值慢：需要先遍历才能找到索引

优点

- 内存占用小，只需要存上一个或下一个的引用

双向链表

双向链表本质还是链表，只不过链表用了一个prev或者next来标记上一个或者下一个对象，而双向链表就是同时拥有prev和next来标记上一个和下一个。除此之外，双向链表还会记录第一个（first）和最后一个节点（last）

缺点

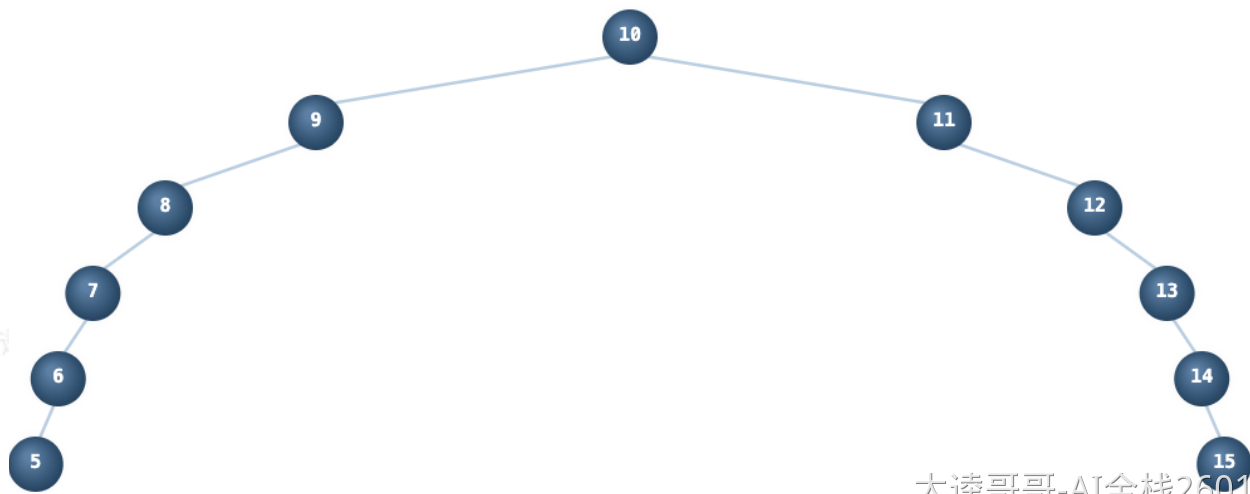
- 内存占用大：存储2个引用，即上一个节点和下一个节点的引用
- 随机获取值慢：需要先遍历才能找到索引

优点

- 遍历：双向遍历，即可以从头到尾或从尾到头进行快速遍历
- 删除尾节点：直接拿到尾节点就可以拿到上一个节点，然后改变上一个节点的next为null即可
- 删除任意节点：直接拿到上一个和下一个节点调整prev和next引用即可，例如A -> B -> C，删除B时，可以直接修改A的next为C，C的prev为A。

二叉树

在双向链表的基础上，把prev改成left，把next改成right，并且规定left比中间值小，right比中间值大，就形成了二叉树结构。



[📎二叉树.html](#)

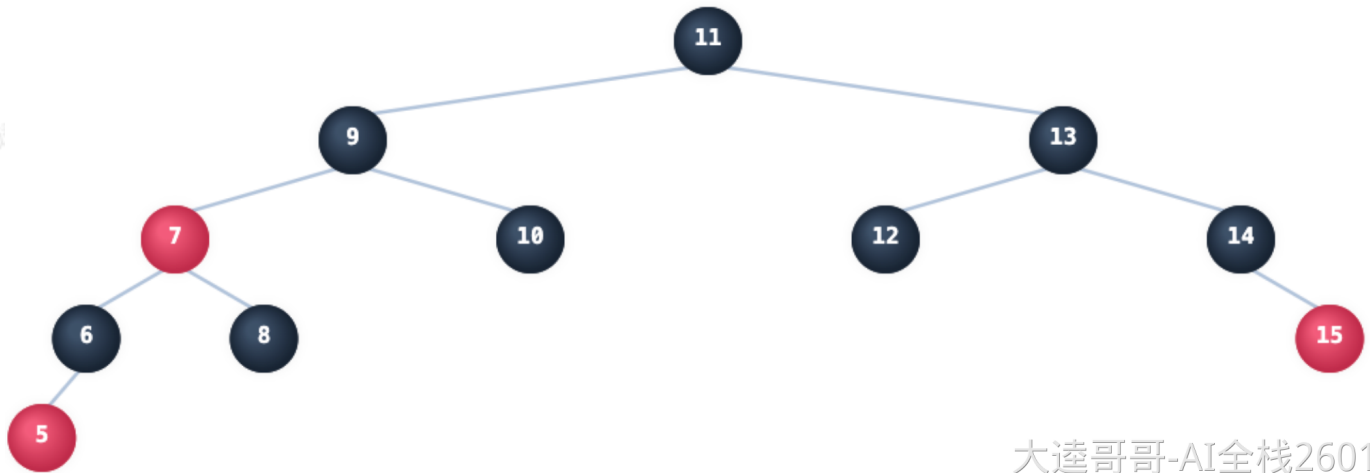
按照左小右大来看，二叉树的访问和插入其实就是一个二分法，比如40、30、50连续插入，那么插入30的时候比40小，就插入到左侧，插入50的时候比40大，就插入到右侧。如果再插入60，那就先找路径。根据40找到50，并且50的右侧没有节点就增加节点。

缺点

- 有可能退化为链表：树容易歪，比如上面的例子中，插入70、80、90、100，都会被插入到右侧。
- 访问速度逐渐变慢：因为退化为链表后，访问最新添加的数据就会越来越慢。

红黑树

红黑树就是在二叉树的基础上，加入红黑元素，控制树不要歪，因为歪了就有可能成为一个链表，导致查询和插入速度降低。



[红黑树.html](#)

红黑树通过加入**红黑颜色标记+旋转**实现了动态平衡。这样查找和插入路径变少，速度就加快了。

逻辑：红黑树最核心的一个逻辑有2点：

1. 从任意节点到任意子节点，经过的黑色节点的数量一致。
2. 节点的最长路径（黑红黑红）不得大于最短路径（黑黑相连）的两倍

这样就可以保证树节点不会过于偏向哪一个方向。当发现树歪了，就将整棵树进行旋转来保持平衡（不是绝对平衡）

优点：

- 存储：接近 $2^K - 1$ ，也就是存下42亿数据，只需要32层（即 2^{32} ），但需要注意的是，并不是可以直接存 $2^K - 1$ ，而是接近，因为并不是每个节点都可以存储到值
- 访问：访问链路短，等同于层级不多的单向链表做一下遍历即可

缺点

- 存在节点浪费问题（二叉平衡树）

集合



用法

自行学习, 能完成以下事项即可

- 构建: new
- 添加值: add、addAll、put、putAll
- 获取值: get
- 遍历值: for、迭代器 (Iterator)
- 获取长度: size